Document 11 — Multi-Agent Systems → Project 4

> This is for self-study and live mentoring. Full plan: CURRICULUM.md

Prerequisites

10_agent_workflows (Project 3 done)

How to Read & Practice This Document

- **What:** splitting one task across several specialized agents.
- **Why:** this is the real goal of the whole curriculum — and also the easiest place in the whole stack to overdo it for no good reason.
- **When:** when one agent's tools or context become too much to handle well, or a task truly needs specialist thinking — never just by default, because multi-agent sounds impressive.
- **How to practice:** 1. Read the pattern table below before writing any code — you should be able to name all 7 patterns from memory before starting. 2. Do the **Basic** paper-design exercise closed-book — guess the cost/speed/reliability before building anything. 3. Do **Intermediate/Real-world** by building two versions of the *same* task, and comparing the real numbers to your guess. 4. Try **Project 4** without looking at any old solution. Let yourself get stuck before asking for a hint. 5. Use **Hint 1** → **Hint 2** only after really trying. Ask for the full solution only if you say **"Show me the solution."** 6. Before moving on, defend out loud — to an imagined tough senior engineer — why this task needed 4 agents and not 1. If you can't, you're not done.

Jump to: Core Concepts · Practice Exercises · Build Task

The Story — what this document is actually building

Project 3 (Doc10) built one agent that could search, reason, pause for approval, and finish. That agent is good at one kind of job. But real work often has several genuinely different jobs stacked on top of each other — go find out what's true, figure out what it means, write it up clearly, and then check the writing is actually good — and asking one agent's single system prompt to be excellent at all four of those at once is asking a lot. That's the problem this document is really about.

The tempting mistake is to assume "more agents" is automatically better, because it sounds more advanced. It usually isn't — a single well-organized agent is cheaper, faster, and has fewer places to

break, and stays your default choice. Multi-agent design only earns its cost when a task truly has separate phases that would fight each other in one prompt, when one agent's tools or history would get overloaded, or when running independent pieces at the same time genuinely saves real time. This document walks through the seven shapes multi-agent systems come in — single, sequential, supervisor, hierarchical, peer-to-peer, parallel, and generator→critic→revision — so that you're choosing a shape on purpose, with a real reason, not by default.

That's the whole story: **the patterns** are just different ways of arranging who talks to whom and in what order. The **`Command` tool** is today's clean way for one agent to say "here's my update, and here's who goes next" in a single step, replacing older, more fragile text-code routing. **Shared state** is the contract the whole team reads and writes, and deciding what belongs in it — versus what stays private to one agent — is a real design skill, not a detail. **Error propagation** decides whether one agent's bad day sinks the whole team or is handled gracefully. And the **Build Task**, Project 4, asks you to put a real version of all of this together: a Supervisor that routes a task to a Research agent, then an Analysis agent, then a Writer, then a Reviewer who can send the work back for changes — a real, working, multi-agent pipeline, not just a diagram of one.

Core Concepts (read this first — everything you need is here)

Topics on this page: The real question: one agent, or many? · Every pattern below: what/why/when/trade-off · The `Command` tool · Shared state design · Logging across multiple agents · Error propagation between agents · Cost and latency · Agent-to-agent communication patterns

The real question: one agent, or many?

Before reaching for any pattern below, ask the one question that decides everything else: does this task genuinely need more than one agent, or would one agent with a bigger, better-organized toolbox do the same job for less money, less delay, and fewer places to break? Picture a small clinic run by one experienced doctor who can also draw blood, read an X-ray, and handle billing. The doctor is slower at each individual task than a specialist would be, but there is only one person to coordinate, one file to read, and nothing gets lost between departments. The clinic only needs to hire more staff once one doctor genuinely cannot keep all of that in their head at once — not because a bigger clinic looks more impressive on the sign outside. Multi-agent systems work the same way: splitting into several agents earns its cost only when a task has genuinely separate phases that would fight each other inside one system prompt, when one agent's tools or history would overload its own reasoning, or when running independent pieces at the same time gives a real, measurable speed win. Reach for more agents because the task actually needs it, never because it sounds more advanced on a resume.

How it really works

- A single agent's system prompt is a shared resource every instruction competes for. Ask it to research, organize, write, and critique in one prompt, and each instruction dilutes the others —

Doc06's "tool description is really a prompt" topic already showed this at the tool-choosing level; here it happens at the whole-task level.

- Tool list size is a second real ceiling. Doc06's tool-selection accuracy degrades as the tool list grows — a 15-tool single agent picks worse than three 4-5-tool specialists would, for the same underlying capability set.
- Context history is a third ceiling. A single agent's scratchpad (Doc07's "scratchpad" topic) grows with every tool call from every phase of the task. A research-then-write-then-review agent's scratchpad by the review phase still holds every dead-end search from the research phase — noise the reviewer has to read past to find the actual draft.
- None of these three ceilings are hit by most tasks. A task with 3-4 tools and a few steps almost never needs splitting — the single-agent column of the table below is the default, not the exception.
- The real test, in order: **(1)** do the task's phases genuinely fight each other in one prompt (research vs. critique are opposite mindsets — one gathers, one doubts); **(2)** would splitting the tools materially improve tool-choice accuracy, not just tidy them up; **(3)** can independent pieces genuinely run at the same time for a real time saving. If none of the three is true, one well-organized agent is the correct, cheaper answer.
- **Why this matters more than memorizing the seven pattern names below:** the failure this whole document exists to prevent is reaching for multiple agents because it sounds advanced, then paying 3-4× the cost and delay of one agent, for a task that never needed splitting. Interviewers ask "why did this need 4 agents and not 1?" specifically because this judgment call, not pattern trivia, is what separates a senior design from a junior one.

Signal	Single agent still wins	Multiple agents earn their cost
Task phases	All phases share one mindset (all "gather," or all "write")	Phases genuinely conflict (gathering facts vs. doubting them)
Tool count	3-6 tools, one agent chooses well	10+ tools total, splitting by job cuts wrong-tool picks
History length	A few steps; scratchpad stays small and relevant	Many steps; an early phase's noise would bury a later phase's signal
Independent work	Steps depend on each other in order	Two or more steps genuinely don't need each other's output first
Your actual evidence	A guess, "it feels like it needs agents"	Measured: you ran both versions and compared real numbers (see Practice Exercises below)

Common mistakes:

- *Mistake:* splitting into agents because the task "sounds complex," without checking whether the phases actually conflict. → *Symptom:* four agents, four times the cost and delay, and the final output is barely different from what one well-prompted agent produced. → *Fix:* run the

Basic/Intermediate/Real-world exercise below on your own task first — build a single-agent version, measured, before writing any multi-agent code.

- *Mistake:* assuming more tools always means more agents. → *Symptom:* a 12-tool single agent that actually performs fine gets split into 4 agents "for cleanliness," adding routing failures with no accuracy gain. → *Fix:* measure the single agent's actual tool-choice accuracy (Doc06's fixture) before assuming the tool list is the problem.

Where you'll meet it: this is Doc07's "When NOT to use an agent" topic, one level up — there the question was "does this need a loop at all?"; here it's "does this loop need to be several loops?" Doc10's entire document is the single-agent proof that phases alone don't force separate agents — its Retriever/Reasoner/Approval steps are three real phases, handled inside **one** LangGraph graph. This document's own Practice Exercises make you measure this decision for real, and Project 4's "Plan Before You Code" step requires you to defend the choice with real numbers before writing a single agent node.

Quick cheat sheet:

- Default to one agent. Split only when phases conflict, tools overload choice, or independent work can run in parallel.
- Three ceilings to check: prompt conflict, tool-list size, scratchpad noise.
- "It sounds more advanced" is not a design reason — it's the exact failure this document guards against.
- Measure, don't guess: build both versions on a real task before deciding.

Every pattern below: what / why / when / trade-off

Once you've decided a task genuinely needs more than one agent, the next decision is *how they're arranged* — who talks to whom, in what order, and who decides what happens next. Think of it like organizing a real team: sometimes the work moves down an assembly line, one person handing off to the next in a fixed order; sometimes a manager decides on the fly who picks up which piece of work; sometimes people just hand things directly to whoever's next, with no manager at all. Each of the seven shapes below is one of those team arrangements, translated into a LangGraph graph. None of them is "the best" one — each is the right tool for a specific shape of task, and picking the wrong one either adds needless routing overhead or forces a task that genuinely needs a decision-maker into a rigid fixed order it doesn't fit.

How it really works

- **Single agent** — one loop, all tools, no routing between agents at all. **What:** Doc07's ReAct loop, unchanged — this "pattern" is really the absence of a pattern, listed here only so the comparison table below has a real baseline. **Why:** simplest to build, cheapest to run, and has the fewest places to break — one scratchpad, one system prompt, one thing to debug. **When:** your default choice, always, until The real question above gives you a real reason not to. **Trade-off:** a long

tool list makes tool choice less reliable, and a long task makes the scratchpad noisy — but for most tasks, neither ceiling is ever hit.

- **Sequential** — agent → agent → agent, in a fixed order, each one's finished output feeding the next as input. **What:** the simplest real multi-agent shape: no router, no decision — just plain edges the whole way (`add_edge("research", "write")`, `add_edge("write", "review")`), Doc09's nodes and edges mechanism with nothing conditional about it. **Why:** keeps a genuinely ordered task clean — research always happens before writing, writing always happens before review, and that order never changes. **When:** the phases are always in the same order and never loop back — a real content pipeline (research → draft → review) is the textbook case. **Trade-off:** total delay is the *sum* of every stage's time, since nothing runs at the same time as anything else — but it's the most reliable and cheapest-to-debug multi-agent shape, because each stage has one narrow, testable job and there's no routing decision that can go wrong.
- **Supervisor** — one router agent (or plain function) decides, at runtime, which specialist runs next, using `Command` to hand off work and update shared state in one step. **What:** a single node whose whole job is reading state and picking the next specialist — everything else is identical to Sequential's specialist nodes. **Why:** handles tasks where the right next step genuinely isn't known ahead of time — a support desk needs different specialists depending on what the customer actually asked, not a rule you can write once and never touch again. **When:** the task type varies per request, and the order isn't fixed — this is Doc10's routing topic ("should I search?"), one level up: now the question is "which agent should handle this?" instead of "should I search?" **Trade-off:** every routing decision is extra time that can itself be wrong — a mis-routed task doesn't crash, it quietly goes to the wrong specialist and comes back with a confidently wrong answer, the same "silent failure, not a crash" shape Doc10 warned about for a router that wrongly skips search. There are three genuinely different ways to write a Supervisor's routing logic, with runnable code right after this list — picking between them is itself a real design decision.
- **Hierarchical** — supervisors of supervisors: a top-level Supervisor routes to *mid-level* supervisors, each of which routes to its own set of specialists. **What:** exactly the Supervisor pattern, nested — a mid-level supervisor and its specialists are usually built and compiled as one Doc09 subgraph, then plugged into the top-level graph as a single node. **Why:** scales the Supervisor pattern past the point where one router can reasonably choose from a long, flat list of specialists — a top-level router choosing between "the writing team" and "the research team" is a much easier decision than choosing directly between 12 individual specialists. **When:** genuinely large systems, with enough specialist areas that a flat list would overload one router's own tool-selection accuracy — rarely worth it below roughly 6-8 specialists. **Trade-off:** every level stacks the Supervisor pattern's own cost, delay, and routing-error risk — a wrong decision at the top sends the whole request to the wrong *team*, and a second routing decision then has to recover in a way a flat Supervisor never has to.
- **Peer-to-peer** — agents hand off directly to each other, using `Command`'s `goto` field to name a specific next agent, with no central router node at all. **What:** each agent's own logic decides who's next, based on its own output — the routing decision moves from one router node into every specialist node individually. **Why:** avoids one router becoming a bottleneck or a single

point of failure, and fits naturally when specialists have clear, non-overlapping conditions for recognizing when it's the *other* agent's turn. **When:** the handoff conditions are genuinely clear and non-overlapping — a triage agent that always hands off to exactly one of two clearly distinct specialists, never both, never neither. **Trade-off:** much harder to reason about and debug than a Supervisor graph — there is no single place to read "the plan," because the routing logic is scattered across every agent's own code instead of living in one router function; Doc09's debugging-a-graph-visually topic still draws the diagram, but the picture alone won't tell you *why* agent A chose agent B over agent C the way one Supervisor function's code would.

- **Parallel** — independent agents run at the same time using Doc09's fan-out/fan-in (built on Doc08b's `asyncio.gather` idea, one layer down), with results merged by a reducer once every branch finishes. **What:** one node's plain edges fan out to two or more specialist nodes at once, all of which fan back in to one shared "combine" node. **Why:** a real, measurable speed win — total wait time is the *slowest* branch's time, not the sum of every branch, the same physics as Doc09's fan-out topic and Doc08b's `gather` one layer down. **When:** the sub-tasks are genuinely independent — if agent B needs agent A's output before it can start, that's Sequential, not Parallel; forcing a dependent step into a parallel branch just means it silently starts with incomplete information. **Trade-off:** merging conflicting or inconsistent results from parallel agents is a real, un-automatic design problem — if a fact-checker and a researcher disagree about the same fact, something in the combine node has to decide whose answer wins, and "just concatenate both" is rarely actually correct.
- **Generator → Critic → Revision** — one agent creates, a separate agent checks it, and the pair loop — using Doc09's loop-back conditional edge — until the critic accepts the work or a hard round limit is hit. **What:** this is Project 4's Writer/Reviewer pair — a generator node, a critic node, and a conditional edge that routes back to the generator on rejection or forward to `END` on acceptance. **Why:** catches quality problems a single creating agent structurally cannot notice about its own output — the same reason a writer needs an editor, not just a re-read of their own draft. **When:** quality genuinely improves from a separate, adversarial-minded check, and "good enough" can be written down clearly enough for the critic to judge consistently — a vague critic ("does this look okay?") makes the loop unpredictable in exactly the way a router with a vague rule does. **Trade-off:** without a hard limit on rounds, this is Doc07's step-limit discipline and Doc09's loop-termination topic all over again, just with two agents taking turns instead of one agent looping alone — and a revision counter that lives in a Python variable instead of state silently forgets its own count across a pause/resume cycle.

Three real ways to write a Supervisor's routing logic — and when to pick each. Doc10's three routing approaches (rules, a small model call, or the main model choosing via a tool) answer "should I search?" A Supervisor answers the same *shape* of question one level up — "which agent should run next?" — and the same real implementations apply.

1. **Rule-based routing (plain Python).** Cheapest, fully predictable, and reads like a checklist.

```
# rule_based_supervisor.py
from langgraph.types import Command
```

```
def supervisor_node(state: TeamState) -> Command:
    if not state.get("research_findings"):
        return Command(goto="research_agent")
    if not state.get("analysis"):
        return Command(goto="analysis_agent")
    if not state.get("draft"):
        return Command(goto="writer_agent")
    revisions_left = state.get("revision_count", 0) < 3
    not_approved = state.get("review_status") != "approved"
    if revisions_left and not_approved:
        return Command(goto="reviewer_agent")
    return Command(goto="__end__")
```

Good when the routing question is really just "which stage is missing its output" — no judgment call, only a checklist of what's already been done.

2. **LLM-judged routing (structured output).** Handles cases where the *right* next step depends on judgment, not just "what's missing."

```
# llm_judged_supervisor.py
from pydantic import BaseModel
from langchain_openai import ChatOpenAI
from langgraph.types import Command
from logging_setup import get_logger              # Doc01's get_logger(name)

logger = get_logger(__name__)

class RoutingDecision(BaseModel):
    next_agent: str
    # one of: "research_agent" / "analysis_agent" / "writer_agent" /
    # "reviewer_agent" / "__end__"
    reason: str

router_model = ChatOpenAI(
    model="gpt-4o-mini", temperature=0
).with_structured_output(RoutingDecision)

def supervisor_node(state: TeamState) -> Command:
    decision = router_model.invoke(
        f"Task: {state['task']}\nDone so far: {state.get('path_log', [])}\n"
        "Which agent should run next?"
    )
    logger.info(
        "supervisor routed to %s (%s)", decision.next_agent, decision.reason
```

```
    )
    return Command(goto=decision.next_agent)
```

Good when the next step genuinely isn't a fixed checklist — for example, deciding whether a task needs Research at all, or can go straight to the Writer because the user already supplied enough facts.

3. **Fixed-order routing with an override.** A hybrid: mostly Sequential, but the Supervisor can skip a stage when state says it's already covered.

```
# fixed_order_supervisor.py
ORDER = ["research_agent", "analysis_agent", "writer_agent", "reviewer_agent"]
STAGE_FIELDS = ["research_findings", "analysis", "draft", "review_status"]

def supervisor_node(state: TeamState) -> Command:
    for field, agent in zip(STAGE_FIELDS, ORDER):
        if not state.get(field):
            return Command(goto=agent)
    return Command(goto="__end__")
```

This ends up nearly identical to rule-based routing above — the honest way to describe it is: **rule-based routing *is* fixed-order-with-an-override**, once the rules are written as a checklist instead of free-form `if` statements. Naming it separately matters once your rules stop being a simple "what's missing" checklist and become a genuine, ordered fallback — try stage 1, and only if stage 1's result looks weak, insert an extra stage before continuing.

Honest comparison:

Approach	Cost	Predictable?	Handles judgment calls?	Pick when
Rule-based (`if`/checklist)	Free, instant	Fully	No	The next step is always "whatever's missing" — no ambiguity
LLM-judged (structured output)	One small model call per routing decision	Mostly — same input usually gives the same route at `temperature=0`	Yes	The right next step depends on the task's content, not just what's already done
Fixed-order with override	Free, instant	Fully	No	Sequential is the real shape, with rare, state-driven exceptions

Full pattern comparison:

Pattern	Fixed or dynamic order?	Extra cost vs. Sequential	Best when
Single agent	N/A — one loop	None (the baseline)	Task has no real phase conflict
Sequential	Fixed	None	Phases always run in the same order
Supervisor	Dynamic	One routing decision per hop	Right specialist varies per request
Hierarchical	Dynamic, nested	A routing decision at every level	Many specialists, grouped into teams
Peer-to-peer	Dynamic, decentralized	Same as Supervisor, spread across agents	Handoff conditions are clear and non-overlapping
Parallel	Fixed branches, concurrent	None extra — often a net time saving	Sub-tasks are genuinely independent
Generator → Critic → Revision	Loop, bounded	One critic call per round	Quality needs a separate, adversarial check

Common mistakes:

- *Mistake:* choosing Hierarchical or Peer-to-peer because they sound more sophisticated than a plain Supervisor. → *Symptom:* a 4-specialist task now has 2-3 extra routing decisions and 2-3 new ways to route to the wrong place, for zero quality gain over one flat Supervisor. → *Fix:* only reach for Hierarchical once a flat Supervisor's own specialist list is genuinely too long to route well (roughly 6-8+); only reach for Peer-to-peer once handoff conditions are provably non-overlapping.
- *Mistake:* using an LLM-judged router for a routing decision that's really just "what's missing" — a checklist dressed up as a judgment call. → *Symptom:* paying for a model call on every single handoff, with routing decisions no more accurate than the free rule-based version would have been. → *Fix:* try rule-based first; move to LLM-judged only once you've actually seen the rules miss real cases, the same discipline Doc10 teaches for its own router.

Where you'll meet it: Doc09's nodes and edges topic is the mechanism every one of these seven shapes is built from — "the supervisor pattern is nothing more than agents are nodes, the supervisor is a conditional edge," in Doc09's own words. Doc10's routing topic is this same rules-vs-model-call decision, one level down, deciding "search or not" instead of "which agent." Project 4 builds the Supervisor pattern for real, with all four specialists; Project 11 — MCPCrew and Project 13 — CodeGuard both reuse this exact pattern table to justify their own architecture choices.

Quick cheat sheet:

- Single agent is the default; the other six patterns are what you reach for only once The real question above gives you a real reason.
- Sequential: fixed order, cheapest multi-agent shape. Supervisor: dynamic order, one router decides. Hierarchical: supervisors of supervisors, for scale. Peer-to-peer: no central router, needs clean handoff conditions. Parallel: independent work at the same time. Generator→Critic→Revision: a bounded quality loop.
- A Supervisor's router is rule-based (free, predictable), LLM-judged (flexible, costs a call), or fixed-order-with-override (a checklist by another name) — pick based on whether the decision is really a judgment call.
- Every pattern beyond Single and Sequential adds a real chance to route to the wrong place — that's the actual price of dynamic decision-making, not a footnote.

The `Command` tool: today's way of handing work between agents

Handing work from one agent to another is really two things happening at once: updating what the team knows (the shared state) and deciding who works on it next (the routing). Doc09 built each of LangGraph's pieces separately — a node returns a state update, a conditional edge decides where to go — and for a while, multi-agent LangGraph code wired those two together by hand, using a made-up text code an agent would return (`"ROUTE_TO_WRITER"`) that a conditional edge then had to parse. `Command` is today's cleaner answer: one typed object that carries both the state update and the destination node, returned directly from the node itself, with nothing to misspell and no separate routing function to keep in sync.

How it really works

- **What `Command` actually is:** an object with two fields — `update` , a dict merged into state exactly like a plain node return value, and `goto` , the name of the next node — returned from a node instead of a plain dict. `Command(update={"draft": text}, goto="reviewer_agent")` does in one line what used to take a node's return value plus a separate conditional edge reading a special field.
- **Why the older text-code pattern was fragile:** the routing signal lived inside a field the agent itself wrote as free text (`state["next_step"] = "ROUTE_TO_WRITER"`), and a conditional edge elsewhere parsed that string. A typo in either the string the agent wrote or the string the edge checked for failed silently — the graph just went to a default path, with nothing telling you why.
- `Command` collapses that into one place: the node that decides the update *is* the node that decides where to go, so there's no second file to keep in sync, and a wrong `goto` value fails the same way a wrong node name in `add_edge` does — loudly, at the point of use, not by silently falling through to a default.
- A node using `Command` doesn't need a separate `add_conditional_edges` call for that hop — the routing lives in the node's own return value. You still register the node with `add_node` , and you still need `add_edge(START, "supervisor")` to get things started, but the hop *out* of the Supervisor is decided per-call, not declared once at graph-build time.

- `goto` can target `"__end__"` to finish the graph, or LangGraph's `Send` primitive (`Command(goto=Send("agent_name", state_update))`) to send a *different* piece of state to a different destination than the graph's overall state — useful for a Supervisor giving one specialist a narrower slice of state than the others need, though most Supervisor patterns don't need this and a plain `Command(goto=..., update=...)` covers the vast majority of real handoffs.
- `Command` composes with everything Doc09 already taught: a node returning `Command` can still be the target of `interrupt()` elsewhere in the graph, and the checkpointer still saves the state `update` half of a `Command` exactly like any other node's return value — nothing about persistence changes.

```
# The old way — a text code, parsed by a separate conditional edge
def writer_agent(state: TeamState) -> dict:
    draft = write_from_findings(state["research_findings"])
    return {"draft": draft, "next_step": "ROUTE_TO_REVIEWER"}  # easy to typo

def route_after_writer(state: TeamState) -> str:
    if state["next_step"] == "ROUTE_TO_REVIEWER":  # must match EXACTLY
        return "reviewer_agent"
    return "END"  # silent fallback on any mismatch

builder.add_conditional_edges(
    "writer_agent", route_after_writer, ["reviewer_agent", END]
)
```

```
# Today's way — Command carries the update AND the destination together
from langgraph.types import Command

def writer_agent(state: TeamState) -> Command:
    draft = write_from_findings(state["research_findings"])
    # one place, nothing to parse
    return Command(update={"draft": draft}, goto="reviewer_agent")
```

Situation	What to do	Why
A node's own logic decides where to go next (Peer-to-peer, or a Supervisor)	Return `Command(update=..., goto=...)`	One typed object; no separate conditional edge to keep in sync
The destination is always the same regardless of what the node found	A plain `dict` return, plus `add_edge(a, b)`	`Command` adds nothing when the next node never varies
You're reading an older LangGraph codebase	Expect a text-code field (`next_step`, `route`) read by a conditional edge	The pattern this document replaces — recognize it, don't copy it into new code

Situation	What to do	Why
A specialist needs a narrower slice of state than the full graph state	`Command(goto=Send("agent", partial_update))`	`Send` targets a *specific* payload at a *specific* destination, not the whole state

Common mistakes:

- *Mistake:* mixing `Command`-returning nodes with a separate `add_conditional_edges` call for the same hop. → *Symptom:* LangGraph raises a compile or runtime error about a node having two competing ways to decide the next step. → *Fix:* pick one mechanism per hop — either the node returns `Command(goto=...)`, or a conditional edge decides, never both for the same edge.
- *Mistake:* copying an old tutorial's text-code routing (`state["next_step"] = "..."`) into new code because that's what older examples show. → *Symptom:* a routing string with a subtle typo silently falls through to a default path, and nothing in the logs explains why the graph didn't go where you expected. → *Fix:* use `Command` for any node whose own logic picks the destination — no string to typo, no second function to keep in sync.

Where you'll meet it: this is Doc09's nodes and edges and state topics, combined into one return value. Project 4's Supervisor is built entirely on `Command`-based handoffs — its whole final step is this mechanism wiring four specialists together. Project 11 — MCPCrew reuses the identical handoff shape with MCP-provided tools inside each specialist.

Quick cheat sheet:

- `Command(update={...}, goto="node_name")` = a state update and a routing decision, in one typed return value.
- Replaces the older "return a text code, parse it in a conditional edge" pattern — recognize the old pattern in legacy code, don't write new code that way.
- Don't mix `Command`-based routing and `add_conditional_edges` for the same hop.
- `Send(...)` targets a specific payload at a specific node — for the rare case where one specialist needs a narrower slice of state than the rest.

Shared state design: what goes in the shared state, and what doesn't

Every multi-agent graph has one shared state object every agent can read, and usually write — Doc09's state topic called it the contract between nodes, and in a multi-agent graph that contract is now between *people*, not just steps. Picture a shared project folder on a team's server: everyone can see the finished reports each person uploads, but nobody wants — or needs — to see the fifty scratch drafts and dead-end searches that led up to each report. The real design question for every field you add to state is the same question that folder answers by its own structure: does every agent genuinely need this, or does it belong in one agent's own private workspace instead?

How it really works

- **Why too much in shared state is a real problem, not just messy:** if the Research agent dumps its whole raw scratchpad — every tool call, every half-formed thought — into shared state, the Writer now has to read past irrelevant noise to find the one paragraph of real findings, and an unrelated agent might accidentally read (or get confused by) another agent's private reasoning. This is Doc07's scratchpad topic applied at team scale: "a multi-agent handoff passes a summary, not the whole log" is exactly the discipline this topic is about.
- **Why too little is just as broken:** if agents only share a one-line final result, a later agent (or a human reviewing the run) can't ask "why did you conclude that" or catch a mistake early — they can only hand off a black box and hope it was right.
- **The actual rule:** shared state holds the task itself, each agent's *finished* output, and routing/status info (whose turn it is, how many revisions have happened) — never a raw tool-call scratchpad, which stays local to the agent node that produced it.
- **Two genuinely different ways to structure this, and picking between them is a real decision:**
- **One shared `TypedDict`, with per-agent namespaced keys. What:** a single state class where each agent owns a small number of clearly-named fields — `research_findings`, `draft`, `review_status` — and reads whichever earlier fields it needs. **Why:** the simplest shape, and it matches exactly how Doc09's checkpointer already works — one dict, saved whole, every superstep. **When:** the default choice for most multi-agent graphs, including Project 4's — a handful of specialists, each contributing one or two clearly-named fields.
- **Separate namespaced sub-states, merged through subgraphs. What:** each specialist (or team, in a Hierarchical system) has its *own* `TypedDict`, built and compiled as its own Doc09 subgraph, and only the fields the parent graph actually needs cross the boundary between the specialist's private state and the shared one. **Why:** keeps a specialist's own internal bookkeeping (a multi-step research agent's own `search_count`, `query`, and `found_chunks` — exactly Doc10's agentic-RAG state shape) fully private, with zero risk of an unrelated field name colliding with another specialist's own field of the same name. **When:** a specialist's own internal logic is complex enough to be a real Doc09 subgraph on its own — a Research agent that runs Doc10's whole multi-step search loop internally is a natural candidate; a Writer that's a single LCEL chain is not.
- **A field that "sounds shared" often isn't.** A Research agent's own `search_count` loop counter (from Doc10's multi-step search) is real state — but it's *that agent's* state, not the team's. Promoting it to the top-level shared `TypedDict` "just in case" is exactly the over-sharing mistake described above, dressed up as thoroughness.
- **This decision has to be made per field, explicitly** — "does every agent need this, or just one?" — rather than defaulting to one big shared object because it's easier to wire up on day one and painful to unwind later, once three agents are already reading a field that was only ever meant for one of them.

```
# Approach 1 — one shared TypedDict, per-agent namespaced keys
from typing import Annotated, TypedDict
import operator

class TeamState(TypedDict, total=False):
    task: str                # what was asked — every agent may read this
    research_findings: str   # Research's FINISHED output — shared
    analysis: str             # Analysis's FINISHED output — shared
    draft: str                # Writer's FINISHED output — shared
    review_status: str          # "approved" / "rejected" — routing, shared
    revision_count: int          # routing info — shared
    path_log: Annotated[list[str], operator.add]
    # ^ who ran, in what order — shared, grows
    # NOT shared: each agent's own raw scratchpad — stays in its own node
```

```
# Approach 2 — a Research agent's own private sub-state, as a Doc09 subgraph
class ResearchSubState(TypedDict, total=False):
    task: str          # shared with the parent — crosses the boundary
    query: str           # PRIVATE — this agent's current search text
    search_count: int     # PRIVATE — this agent's own loop counter (Doc10)
    found_chunks: list[dict]   # PRIVATE — raw results, not yet summarized
    research_findings: str      # shared — the only field needed back

# a whole Doc10-style agentic RAG graph
research_subgraph = research_builder.compile()
# plugged in as ONE node, like any other
team_builder.add_node("research_agent", research_subgraph)
```

Only `task` and `research_findings` cross the boundary between `ResearchSubState` and `TeamState` — `query`, `search_count`, and `found_chunks` never leave the subgraph, so the Writer never sees them and can't be confused by them.

Situation	What to do	Why
Every downstream agent needs it	Put it in the shared `TypedDict`	That's exactly what shared state is for
Only one agent's own reasoning needs it, and it's simple	A local variable inside that agent's node	Never checkpointed, never seen by anyone else — nothing to leak
One agent's own internal loop is complex (multi-step search, its own retry logic)	A private sub-state, wrapped as a Doc09 subgraph	Keeps that agent's bookkeeping fully separate, even from field-name collisions

Situation	What to do	Why
A field name could plausibly mean two different things to two agents	Rename it to be agent-specific, or keep it in a sub-state	An ambiguous shared field is exactly how one agent overwrites another's meaning by accident
A large raw result (a full research dump, a full document)	Keep it in the producing agent's own state/subgraph; share only the summary	The checkpointer saves the whole state every superstep — bulk here is a real cost, Doc09's own state-size warning

Common mistakes:

- *Mistake:* putting a Research agent's own raw tool-call scratchpad directly into the shared `TeamState`. → *Symptom:* the Writer's prompt is now stuffed with unrelated search noise, and it either gets confused or has to be told explicitly to ignore most of what it can see. → *Fix:* keep the raw scratchpad private to the Research agent's own node (or subgraph); share only the finished `research_findings` field.
- *Mistake:* two agents each add a field with the same name but a different meaning ("status" meaning "approved" to the Reviewer and "in progress" to the Writer). → *Symptom:* one agent silently overwrites the other's meaning, and a later node reads a value that doesn't mean what it thinks. → *Fix:* name fields for what they hold, not just what sounds convenient — `review_status` and `writer_status`, never one shared `status`.

Where you'll meet it: this is Doc09's state topic — "state is the contract between agents" — made concrete with real field names. Doc10's state-must-carry-what-was-found topic is this same question for one agent, one level down — a research agent's found chunks belong in *its own* state the same way an approval-pending draft belonged in Doc10's single-agent state. Project 4's `state.py` is Approach 1 above, grown by one field per specialist; Project 5 is where a genuinely complex specialist first gets promoted to Approach 2's subgraph shape.

Quick cheat sheet:

- Shared state: the task, each agent's *finished* output, and routing/status info. Never a raw scratchpad.
- Decide per field, explicitly — "does every agent need this?" — never by defaulting everything to shared.
- One shared `TypedDict` with namespaced keys is the default; a private sub-state (Doc09 subgraph) is for a specialist complex enough to need its own bookkeeping.
- A field name that could mean two things to two agents is a bug waiting to happen — name for meaning, not convenience.

Logging across multiple agents: which agent logged what

Doc01's `get_logger(name)` gives every module its own named logger, and that's already enough for a single agent — one process, one logger, one clear trail. The moment a second agent joins the

run, that trail gets a real new problem: every agent might use the exact same model and the exact same tool, and log lines that look almost identical, so a log full of `INFO: calling search_documents` tells you nothing about *which* agent called it, or which of ten concurrent runs it belongs to. Solving this isn't optional once you have more than one agent — it's the difference between a debuggable system and a pile of indistinguishable log lines.

How it really works

- **The two things a multi-agent log line needs that a single-agent one didn't:** which *agent* logged it, and which *run* it belongs to (a thread/request id) — without both, two agents' interleaved log lines in a shared file or console are impossible to untangle.
- **Which agent:** give every specialist its own named logger, following Doc01's exact naming convention — `get_logger(f"agents.{agent_name}")` — so `%(name)s` in the formatter (Doc01's own recommendation) already shows which agent wrote each line, with zero extra fields to add by hand.
- **Which run:** a `run_id` (or reuse Doc09's own `thread_id`) needs to appear on every line from every agent in that run — the cleanest way is a `logging.Filter` that reads a `contextvars.ContextVar` set once at the start of `graph.invoke(...)`, so every logger, in every agent, in that one call, gets the id attached automatically, with no agent's own code having to remember to pass it along.
- This is genuinely the same discipline Doc01's logging topic already taught — one format, one place configured — just with one more field (`run_id`) that only becomes necessary once more than one thing can be running, and more than one agent can be logging, at the same time.
- Log the **routing decision itself**, every time, at `INFO` — which agent the Supervisor chose and, if it's an LLM-judged router, why. This is the single most useful line for debugging a mis-routed task after the fact, and it costs one `logger.info()` call.

```
# multi_agent_logging.py — per-agent logger + a run id on every line
import contextvars
import logging
from logging_setup import get_logger          # Doc01's get_logger(name)

_run_id = contextvars.ContextVar("run_id", default="-")

class RunIdFilter(logging.Filter):
    def filter(self, record: logging.LogRecord) -> bool:
        record.run_id = _run_id.get()
        return True

def start_run(run_id: str) -> None:
    _run_id.set(run_id)   # call this ONCE, right before graph.invoke(...)

# each agent gets its own named logger — %(name)s shows which agent wrote it
research_logger = get_logger("agents.research")
```

```
writer_logger = get_logger("agents.writer")

# formatter (configured once, Doc01's rule):
# "%(asctime)s [%(run_id)s] %(name)s: %(message)s"
research_logger.info("found %d chunks for query: %s", len(chunks), query)
# -> 2026-01-18 10:03:41 [run-881] agents.research: found 4 chunks for
#    query: ...
```

Situation	What to do	Why
One agent, one run at a time	Doc01's plain `get_logger(__name__)` is enough	Nothing to disambiguate yet
Several agents, one run at a time	A named logger per agent (`agents.research`, `agents.writer`)	`%(name)s` already shows which agent, no extra code
Several runs at once (a real server)	A `run_id` / `thread_id` on every line, via a `Filter` + `ContextVar`	Interleaved log lines from different runs are otherwise impossible to separate
A routing decision	Always log it at `INFO` — who was chosen, and why if it's LLM-judged	The single most useful line for debugging a wrong route later

Common mistakes:

- *Mistake:* every agent logging through the same generic logger name (`get_logger("agent")` everywhere). → *Symptom:* a log file full of identical-looking lines with no way to tell which specialist did what. → *Fix:* one logger name per agent, following the exact module-naming convention Doc01 already teaches — `agents.research`, `agents.writer`, not one shared name.
- *Mistake:* passing a `run_id` manually as a function argument through every single node. → *Symptom:* it's easy to forget on a new node, and the moment one node forgets it, that node's lines are unattributable. → *Fix:* a `ContextVar` set once at the start of the run, read automatically by a logging `Filter` — no node has to remember anything.

Where you'll meet it: this is Doc01's logging topic — "give each agent its own named logger... or a multi-agent log is unreadable" — made concrete with real code. Project 4's own requirement for "a visible log of which agents ran, in what order, and why the supervisor routed the way it did" is exactly this pattern, built for real. Doc13 turns these same per-agent, per-run log lines into trace spans.

Quick cheat sheet:

- One named logger per agent — `agents.<name>`, following Doc01's convention.
- A `run_id` / `thread_id` on every line once more than one run can happen at once — via a `ContextVar` and a `Filter`, not a manually threaded argument.
- Always log the Supervisor's routing decision, and why, at `INFO`.

- The goal: read one log line and know which agent, on which run, did what — with zero guessing.

Error propagation between agents

Doc01 taught error handling for one function: catch a specific error, then retry, fall back, or raise a clear message up to the caller. Doc02 grew that into a real retry-with-backoff discipline for one outbound call. In a multi-agent pipeline, the same problem shows up one level bigger: when one agent fails outright, or succeeds but returns something unusable — an empty draft, a nonsense analysis — what happens to the *whole team's* work, not just that one function call? Picture a relay race where one runner drops the baton. Do you stop the race entirely, let that runner pick it back up and keep going, or send the next runner around to grab it from the ground and carry on? All three are real, defensible choices — the mistake is never deciding on purpose and just hoping the baton somehow makes it to the finish line anyway.

How it really works

- **The tempting, wrong default:** letting a bad result quietly flow downstream. A Writer that receives an empty `research_findings` field will still cheerfully write *something* — and now you have a confident-sounding final answer built on nothing, which is much harder to catch than a loud crash. This is Doc02's "200 doesn't mean safe to trust" idea, one level up: an agent handoff "succeeding" (no exception thrown) says nothing about whether what it handed off is actually usable.
- **Three real options, honestly compared, with runnable code for each:** 1. **Fail the whole run. What:** the failing agent's node raises a named exception (Doc01's custom-error discipline — `ResearchAgentError`, not a bare `Exception`), and nothing downstream ever runs. **Why:** the safest option — no agent ever acts on bad input from a failed step. **When:** an early, foundational step fails (Research finds nothing at all) and every later stage genuinely depends on it — continuing would just waste more calls building on a broken foundation. 2. **Let the Supervisor decide (retry or route around). What:** the failing agent returns a clear, structured failure signal instead of raising, and the Supervisor's own routing logic checks for that signal and decides: retry the same agent (maybe with a different prompt), skip it and continue with a partial pipeline, or escalate to a full failure. **Why:** most flexible — the decision happens with full knowledge of the whole task's state, not just the one failing agent's local view. **When:** the failure might be temporary or fixable (a flaky search, a tool timeout) and it's genuinely unclear in advance whether stopping or continuing is right — letting the Supervisor look at the actual situation beats guessing at graph-design time. 3. **Per-agent retry with backoff. What:** the agent's own node wraps its own risky call in Doc02's backoff loop before ever reporting failure upward — the agent tries to recover on its own first. **Why:** catches genuinely transient problems (a network blip, a rate limit) without ever bothering the Supervisor or the rest of the team. **When:** the failure type is a known, common, likely-transient one — exactly Doc02's own retry table (timeouts, 429s, 5xxs) — not a fundamental problem with the task itself, which no amount of retrying will fix.
- **These three compose, they don't compete.** A real pipeline usually retries locally first (option 3), reports a structured failure if retries are exhausted (feeding option 2), and only escalates to a

full stop (option 1) if the Supervisor's own routing logic decides the task genuinely can't continue without that agent's output.

- **The check that makes any of this possible:** the Supervisor (or a dedicated check after each agent) has to actually look at what an agent returned, not just assume it worked because no exception was thrown — the exact "did this actually succeed" check Doc01 taught for one function, now applied to every handoff in the pipeline.

```
# Option 1 — fail the whole run
class ResearchAgentError(Exception):
    """Raised when Research cannot produce usable findings at all."""

def research_agent(state: TeamState) -> Command:
    findings = run_research(state["task"])
    if not findings:
        task = state["task"]
        raise ResearchAgentError(f"No usable findings for task: {task!r}")
    return Command(
        update={"research_findings": findings}, goto="analysis_agent"
    )
```

```
# Option 2 — let the Supervisor decide
def research_agent(state: TeamState) -> Command:
    findings = run_research(state["task"])
    if not findings:
        return Command(
            update={
                "research_error": "no usable findings",
                "path_log": ["research:failed"],
            },
            goto="supervisor",
        )
    return Command(update={"research_findings": findings}, goto="supervisor")

def supervisor_node(state: TeamState) -> Command:
    if state.get("research_error"):
        if state.get("research_retry_count", 0) < 1:
            # one retry
            return Command(
                update={"research_retry_count": 1}, goto="research_agent"
            )
        return Command(
            update={"path_log": ["supervisor:skipped_research"]},
            goto="writer_agent",
```

```
        )
    # ...normal routing continues
```

```
# Option 3 — per-agent retry with backoff, before ever reporting up
from http_client import request_with_retry     # Doc02's Build Task
from exceptions import TransientHTTPError      # Doc02's Build Task

def research_agent(state: TeamState) -> Command:
    params = {"q": state["task"]}
    try:
        results = request_with_retry("GET", search_url, params=params)
    except TransientHTTPError:
        return Command(
            update={"research_error": "search unavailable"}, goto="supervisor"
        )
    findings = summarize(results)
    return Command(update={"research_findings": findings}, goto="supervisor")
```

Situation	What to do	Why
A known, likely-transient failure (timeout, rate limit)	Retry locally, inside the agent's own node (Option 3)	Recovers without ever involving the rest of the team
Retries exhausted, or an unclear/task-specific failure	Report structured failure to the Supervisor (Option 2)	The Supervisor has the whole task's state to decide with
A foundational step fails and nothing downstream can proceed without it	Raise and fail the whole run (Option 1)	Continuing would waste calls building on a broken foundation
An agent "succeeds" but returns something empty or nonsense	Treat it as a failure anyway — check the *content*, not just the absence of an exception	The exact "200 isn't safe to trust" trap, one level up

Common mistakes:

- *Mistake:* letting a bad result (an empty draft, a nonsense analysis) flow downstream silently because no exception was raised. → *Symptom:* a confident-sounding final answer built on nothing, much harder to catch than a crash. → *Fix:* check the actual *content* of every handoff, not just whether the call completed — Doc01's "did this actually succeed" check, applied at every agent boundary.
- *Mistake:* retrying at three layers at once — the agent's own node, the Supervisor's retry logic, and an outer wrapper — with no coordination. → *Symptom:* one bad minute turns into far more calls than intended, echoing Doc02's "retries stack" warning, now with agents instead of HTTP

calls. → *Fix:* pick exactly one layer to own retries for a given failure type, and set the others to zero retries for that same failure.

Where you'll meet it: Doc01's errors topic is the one-function version of Option 1; Doc02's retry-with-backoff topic is exactly Option 3, reused unchanged inside one agent's node. Project 4's reviewer agent that "can reject the writer's work... with a limit on how many revision rounds can happen" is Option 2's decision logic, specialized to one particular kind of "failure" (a rejected draft, not a crashed agent). Project 5 hardens this into real production error handling across the whole team.

Quick cheat sheet:

- Three real options: fail the whole run, let the Supervisor decide, or retry locally first — they compose, they don't compete.
- Check content, not just "no exception was thrown" — an agent can "succeed" and still hand off something useless.
- Named exceptions (Doc01) for hard stops; a structured failure field in state for anything the Supervisor should decide about.
- Pick exactly one layer to own retries for a given failure type — stacking retries at every layer multiplies cost, it doesn't add safety.

Cost and latency: multi-agent is not free

Every extra agent in a pipeline is at least one more paid model call — often several, if that agent uses tools or loops internally — which means real added cost and real added delay, since each call has to finish before the next agent can start, unless you're specifically running things in parallel. A 4-agent pipeline isn't "roughly the same speed" as a single agent doing the whole task in one pass; it can easily run 3-4× the cost and wall-clock time, and the only way to know your specific number is to measure it, not guess.

How it really works

- **When the overhead is worth it:** the task genuinely has separate phases that would conflict or get confused if forced into one system prompt; specialist framing measurably improves quality (a dedicated Reviewer catching real mistakes a Writer wouldn't catch in its own output); or independent pieces can run at the same time and the Parallel pattern's speed-up outweighs the extra calls.
- **When it isn't:** this is the same judgment call Doc07 asked one level down — Doc07 asked "does this task need an agent loop at all, or is one tool call enough?"; here the question is "does this task need *multiple* agents, or would one well-organized agent with more tools do the same job for a fraction of the cost?" Both questions share the same trap: reaching for more machinery because it sounds more advanced, not because the task actually needs it.
- **The budget has to be global, not per-agent** — this is Doc07's step-limit topic making the same point directly: "a 5-agent system with a 10-step cap each has an effective ceiling of 50 model

calls, and if one agent can call another as a tool, that ceiling multiplies instead of adding." A single `total_calls` counter in the team's shared state, checked by the Supervisor before every handoff, is the fix — not five separate, uncoordinated per-agent counters.

- **Layer the same three limits Doc07 and Doc09 already teach, at the team level instead of the single-agent level:** a total step/call count across the whole team, a wall-clock deadline for the whole run, and a token/cost budget for the whole run — each catches a failure the others miss, and "one agent looped internally" and "the team looped as a whole" are two different failure modes that need two different counters.
- **How to actually know, instead of guessing:** measure it. Build the same task as a single-agent version and a multi-agent version, run both on the same test inputs, and compare real token counts and real wall-clock time — the Practice Exercises below are built around exactly this comparison.

```
# A global call budget, checked by the supervisor before every handoff
MAX_TOTAL_CALLS = 20

def supervisor_node(state: TeamState) -> Command:
    if state.get("total_calls", 0) >= MAX_TOTAL_CALLS:
        return Command(
            update={"path_log": ["supervisor:budget_exceeded"]}, goto="__end__"
        )
    # ...normal routing continues; each agent node increments total_calls by 1
```

Situation	What to do	Why
A task's phases genuinely conflict in one prompt	Pay the multi-agent overhead	Quality gain outweighs the extra cost
Independent sub-tasks that can run at the same time	Use Parallel — the overhead is often a net time saving	Total wait is the slowest branch, not the sum
A task with no real phase conflict, no tool overload	Stay single-agent	3-4× the cost/delay for no real quality gain
Five agents, each with its own 10-step cap	Replace with one shared, team-wide budget	Per-agent caps add up (or multiply) into a much bigger real ceiling than anyone intended
You're not sure if the overhead is worth it	Measure both versions on the same real inputs	This document's own Practice Exercises are built for exactly this

Common mistakes:

- *Mistake:* giving every agent its own separate step/call limit, assuming the limits "add up to something reasonable." → *Symptom:* a run that should have stopped at 15 calls quietly reaches

50, because five separate 10-step caps never talked to each other. → *Fix:* one shared counter in team state, checked by the Supervisor before every handoff — Doc07's "budgets must be global" rule, applied for real.

- *Mistake:* assuming a 4-agent pipeline "can't be that much slower" without ever measuring it. → *Symptom:* a production system that's 3-4× slower and pricier than a single well-prompted agent would have been, discovered only after the fact from a billing surprise. → *Fix:* build and measure both versions before committing to the multi-agent design — exactly this document's Basic/Intermediate/Real-world exercise sequence.

Where you'll meet it: Doc07's step-limit topic is the single-agent version of this exact budget discipline — "in a multi-agent system, budgets must be global... [Doc11] is where the shared budget lives" is Doc07's own forward reference to this section. Project 4's "Plan Before You Code" step requires exactly this measured comparison before any agent code gets written, and its own Failure exercise (`convergence_and_cost_cutting`) has you cut a working 4-agent run's cost by 30%+ under a real constraint.

Quick cheat sheet:

- Multi-agent overhead is real: often 3-4× the cost and delay of one well-prompted agent.
- Worth it when phases genuinely conflict, a specialist measurably improves quality, or independent work runs in parallel.
- One shared, team-wide call/step budget — never five separate per-agent caps that silently add up.
- Measure the real numbers before committing to a multi-agent design — don't guess.

Agent-to-agent communication patterns beyond the `Command` tool

This document focuses on the Supervisor pattern — one router deciding who goes next, using `Command` to hand off work and update state in one step. That's a genuinely common, reliable, and teachable shape, but it isn't the only one real multi-agent systems use, and recognizing the others is part of the same judgment this whole document teaches.

How it really works

- **A blackboard pattern** has no central router at all: every agent reads from and writes to one shared message log that all agents can see, and each agent decides for itself, by looking at that log, whether it's its turn to act. Useful when the right next step genuinely depends on the accumulated state of the whole conversation rather than one router's single decision — closer to a group chat than a chain of command.
- **A fixed pipeline** (already named above as the Sequential pattern) goes the other direction entirely: agent A's output pipes straight into agent B as input, in a fixed order, with no routing decision made by anyone, human or agent. Simplest to reason about, and only works when the order truly never changes.

- **Why it's worth knowing these exist, even without building them here:** Supervisor-routing is what this document teaches because it's the clearest to learn from and the most common in production LangGraph code — but "multi-agent system" doesn't automatically mean "Supervisor." Recognizing when a blackboard or a fixed pipeline actually fits a task better than adding a router is part of the same judgment The real question: one agent, or many? asks at the very top of this document.

```
# blackboard.py — no central router; each agent checks the shared log itself
class BlackboardState(TypedDict, total=False):
    log: Annotated[list[dict], operator.add]   # every agent's posts, in order

def researcher_agent(state: BlackboardState) -> Command:
    already_posted = False
    for entry in state.get("log", []):
        if entry["from"] == "researcher":
            already_posted = True
    if already_posted:
        return Command(goto="writer_agent")  # someone else's turn now
    findings = run_research(state["log"][-1]["text"])
    post = {"from": "researcher", "text": findings}
    return Command(update={"log": [post]}, goto="writer_agent")
```

Each agent's own check ("have I already posted? does the log contain what I need?") replaces the Supervisor's single routing function — which is exactly why a blackboard is harder to debug: there's no one function you can read to see "the plan."

Pattern	Who decides "who's next"?	Best when
Supervisor (this document's focus)	One router node	The right specialist varies, and you want one readable place to see the routing logic
Blackboard	Each agent, reading the shared log itself	The right next step depends on the whole conversation's accumulated state, not one clean decision point
Fixed pipeline (Sequential)	Nobody — the order is fixed at graph-build time	The order genuinely never changes

Common mistakes:

- *Mistake:* assuming every multi-agent system needs a Supervisor, because that's the pattern most examples show. → *Symptom:* building a router for a task that was always going to run in the exact same fixed order — real overhead for a decision that was never actually in question. → *Fix:* check The real question and the pattern table above first; a fixed pipeline needs no router at all.

- *Mistake:* reaching for a blackboard pattern for its own sake, without a real reason no Supervisor could handle. → *Symptom:* a system with no single place to read "the plan," debugged by reading every agent's own logic instead of one router function. → *Fix:* default to Supervisor; reach for a blackboard only once a router genuinely can't capture the real decision.

Where you'll meet it: this rounds out the pattern table from Every pattern below with two shapes this document doesn't build further, but that Project 11 — MCPCrew and Project 13 — CodeGuard both reference when justifying why they picked Supervisor over an alternative.

Quick cheat sheet:

- Supervisor: one router, one readable place to see the plan — this document's focus.
- Blackboard: no router, every agent reads the shared log itself — for when the right next step genuinely depends on accumulated context.
- Fixed pipeline: no routing decision at all — for when the order never changes.
- Default to Supervisor; the other two are for the specific cases where it genuinely doesn't fit.

Go Deeper (Optional)

You don't need any of these to understand the Core Concepts above — use them if you want a second explanation or more detail.

- LangGraph documentation — multi-agent section — search for "multi-agent," "supervisor," and "Command" in the docs.
- LangChain blog — search for multi-agent architecture posts; read 1-2 recent ones for current patterns.
- Anthropic — Building Effective Agents — read the orchestrator-worker / evaluator-optimizer sections again now that you can match them directly to the patterns above.

Practice Exercises

Setup for this document's practice code: work inside `11_multi_agent_systems/` (same venv as before — if it's not active, `source .venv/bin/activate`). New packages for this document: `pip install langgraph langchain-openai`.

Where your code lives: all of it under `11_multi_agent_systems/practice/` (`mkdir -p practice`), never loose beside this README. Exercises are grouped **by topic, not by difficulty level** — the same convention as Doc01/02/07/09/10 — so one topic's growth from basic to intermediate stays visible in one file.

The full file layout, all exercises:

```
practice/
├── architecture_comparison_practice.py  Basic + Intermediate +
│                                        Real-world (3 sections)
├── convergence_and_cost_cutting_practice.py  Failure
└── supervisor_routing_practice.py       Edge cases
```

Why each script exists:

- `architecture_comparison_practice.py` — the same task, designed on paper then built two ways (sequential, supervisor) and measured — the comparison every later "why N agents" decision in this curriculum traces back to.
- `convergence_and_cost_cutting_practice.py` — a revision loop that hits its limit cleanly, plus a real, measured cost cut — both things Project 4's reviewer loop needs.
- `supervisor_routing_practice.py` — proves a supervisor can route correctly (and recover) when two specialists genuinely overlap, not just when the order was never in question.

How to run each exercise: if two exercises below are really about the same thing, save them together in ONE script named after that topic — for example, if two exercises are both about `.env` config, save both in one file like `env_config_practice.py`, with each level's version as its own clearly labeled section inside it. Run each topic's file directly, for example: `python practice/env_config_practice.py`.

For this document:

- Basic (`paper_design`), Intermediate (`sequential_measure`), and Real-world (`supervisor_compare`) are all the same running comparison — design the single/sequential/supervisor options on paper, then build and measure the sequential version, then build and measure the supervisor version against it — save all three together as `practice/architecture_comparison_practice.py`, with each level as its own section.
- Edge cases (`ambiguous_routing`) asks a different question (can the supervisor route correctly when two specialists overlap) — save it as its own topic, `practice/supervisor_routing_practice.py`.
- Failure (`convergence_and_cost_cutting`) is its own topic — save it as `practice/convergence_and_cost_cutting_practice.py`.

Why group by topic instead of by level: if you save each exercise by difficulty level instead, the different versions of the same idea end up scattered across separate files, and you can never see how one topic grows from simple to harder in one place. Grouping by topic keeps that growth visible — open one file, and you see the whole journey for that one thing, from basic to intermediate, side by side.

Jump to an exercise: Basic · Intermediate · Real-world · Edge cases · Failure · Build Task

Basic — design on paper before touching code

- **What:** design the same task 3 ways (single agent / sequential / supervisor) on paper, guessing which wins on cost/speed/reliability, before writing any code.
- **Why:** this document's central skill is judgment, not syntax — practicing the judgment call *before* code exists is the only way to actually test your intuition against reality afterward.
- **When you'll hit this for real:** the start of every real multi-agent project, including Project 4's own Architecture-Before-Code step.
- **How to code it:** in the `# Basic` section, write — as comments — one paragraph per design (single/sequential/supervisor) describing the flow, then a one-line prediction for cost, speed, and reliability for each. Use The real question and the pattern comparison table above to argue your prediction, not just guess it.
- **Save as:** `practice/architecture_comparison_practice.py`, under a `# Basic` section (this file also holds the Intermediate and Real-world exercises below, each in its own section).
- **Builds on:** The real question: one agent, or many? and the pattern comparison table above — you're using exactly those to make your on-paper prediction.
- **Used later by:** the Intermediate and Real-world exercises below (same file), and Project 4's own "Plan Before You Code" step.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Intermediate — measure the sequential version

- **What:** build the sequential version of a 2-step task (like research → write) and measure real cost/speed against your Basic-level guess.
- **Why:** comparing your prediction to the real number is what actually calibrates your judgment for next time — skipping this step means you never find out if your intuition was right.
- **When you'll hit this for real:** any time you're deciding, for a real project, whether a fixed pipeline is the right shape.
- **How to code it:** two functions chained directly (`write(research(topic))`), with token usage and wall-clock time printed for each stage and the total.
- **Save as:** `practice/architecture_comparison_practice.py`, under an `# Intermediate` section (this file also holds the Basic and Real-world exercises, each in its own section).
- **Builds on:** the Basic on-paper sequential design in this same file.
- **Used later by:** the Real-world exercise below (same file), which compares its measured numbers against this one.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Real-world — build and compare the supervisor version

- **What:** build the supervisor version of the same task and compare it against the sequential one on the same test inputs.
- **Why:** this is the direct, apples-to-apples comparison that lets you actually answer "why 4 agents and not 1" with real numbers instead of a guess.

- **When you'll hit this for real:** this document's own Architecture-Before-Code step, which asks for exactly this comparison before Project 4 begins.
- **How to code it:** a small LangGraph with a supervisor node routing to the same two functions as nodes (try the rule-based router first), run on the same 3 test inputs as the sequential version, comparing cost/speed/output quality.
- **Save as:** `practice/architecture_comparison_practice.py`, under a `# Real-world` section (this file also holds the Basic and Intermediate exercises above, each in its own section).
- **Builds on:** The `Command` tool and its rule-based routing example above, and the Intermediate measurement in this same file.
- **Used later by:** Project 4's "Plan Before You Code" step, and the cost and latency topic above.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Edge cases — an ambiguous routing decision

- **What:** a supervisor task where two specialists could both reasonably apply — check whether the supervisor routes correctly, and what happens when it doesn't.
- **Why:** ambiguous routing is where supervisor-pattern systems actually break in production — you need to have watched it happen once, deliberately.
- **When you'll hit this for real:** Project 4's Research vs. Analysis boundary, where a task could plausibly need either first.
- **How to code it:** write a supervisor with two specialists whose descriptions slightly overlap, run 5 ambiguous test prompts through it, and log which one got picked each time (using the per-agent logging pattern above).
- **Save as:** `practice/supervisor_routing_practice.py`.
- **Builds on:** the three Supervisor routing approaches above — try the rule-based router first, then the LLM-judged one, and compare how each handles the same ambiguous prompts.
- **Used later by:** Project 4's Research vs. Analysis boundary.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Failure — a loop that won't converge, and a cost-cutting pass

- **What:** build a generator↔critic loop with no guarantee it will ever agree, and watch it fail to stop (carefully, with a real limit in place). Then, separately, try to cut a 4-agent run's cost or speed by 30%+ without breaking correctness.
- **Why:** both are real production concerns — a revision loop that never converges is a direct threat to your uptime, and cost-cutting under a real constraint is a skill interviewers specifically ask about.
- **When you'll hit this for real:** Project 4's Writer/Reviewer pair, and any real system operating under a cost budget.
- **How to code it:** give the critic impossible-to-satisfy criteria, run the loop with a low hard limit, and confirm it exits with a clear "couldn't converge" report. Then profile a working 4-agent run for

the most expensive step, and try cutting it (shorter prompts, a cheaper model for one agent, caching) while re-running your test set to confirm correctness held.

- **Save as:** `practice/convergence_and_cost_cutting_practice.py`.
- **Builds on:** the Generator → Critic → Revision trade-off above, and the cost and latency topic's global-budget discipline.
- **Used later by:** Project 4's Writer/Reviewer pair, and any real system operating under a cost budget.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Build Task — Project 4: Multi-Agent System

Stuck on the Build Task? Hint 1 · Hint 2 · Show me the solution

Note: this Build Task is not a `practice/build_task/` folder. It **is** Project 4 — ContentForge, the same way Doc10's Build Task **was** Project 3. The step-by-step build guide lives in project_4_contentforge_multi_agent/READING_README.md — follow that, not just the summary below.

Goal: Supervisor → Research Agent → Analysis Agent → Writer Agent → Reviewer Agent, with routing, shared state, and a checker/reviewer step that can send work back for changes.

Requirements:

- A supervisor node that routes to the right specialist(s) based on the task's state, using the `Command` tool for handoffs.
- Shared state that each agent reads and writes, without it filling up with each agent's own unrelated notes (decide what's shared vs. private — see Shared state design above).
- A reviewer agent that can reject the writer's work and send it back for changes, with a limit on how many revision rounds can happen (no endless generator↔critic loop).
- A clear, on-purpose choice for what happens when one specialist fails or returns something unusable — see Error propagation between agents above.
- A visible, per-agent, per-run log of what happened and why — see Logging across multiple agents above.
- Full LangGraph saved-state, carried over from Project 3.

Inputs: a free-text research/writing task (like "research X and write a short brief").

Outputs: a final, reviewed piece of writing, plus a visible log of which agents ran, in what order, and why the supervisor routed the way it did.

Constraints: no two agents should redundantly call the same tool for the same sub-task; the revision loop must have a limit, and must end with either an accepted result or a clear "couldn't agree" report.

Suggested files:

```
project_4_contentforge_multi_agent/
├── agents/
│   ├── analysis_agent.py   figures out what the research means
│   ├── research_agent.py   looks things up
│   ├── reviewer_agent.py   checks the draft, can send it back
│   ├── supervisor.py       routing logic, using Command
│   └── writer_agent.py     drafts the piece
├── graph.py              wires all 5 nodes into a StateGraph
├── llm.py                 the one chat model every agent imports
├── main.py              entry point, runs the graph
├── state.py               the shared state shape
└── test_project4.py     checks the recoverable and never-agrees cases
```

Why each file exists:

- `agents/*.py` — one file per specialist, so each agent's own logic is testable and readable on its own, separate from the routing.
- `graph.py` — builds on the same `StateGraph` / `Command` mechanics `supervisor_compare` already practiced, now with 5 nodes instead of 2.
- `llm.py` — one `model = ChatOpenAI(...)` line every agent imports, so switching the model for all five agents is a one-line change.
- `main.py` — the one place a task actually gets run, so the graph-building and the running-it concerns stay separate.
- `state.py` — one shared shape every agent reads and writes, so `research_findings`, `analysis`, `draft`, and `review_status` all mean the same thing everywhere.
- `test_project4.py` — proves the revision loop actually recovers on rejection, and actually stops at the limit when it never agrees — the same discipline `convergence_and_cost_cutting` already trained.

Functions/Components to build:

- `agents/supervisor.py` → routing logic using `Command`
- one file per specialist agent, each a graph node (or sub-graph)
- `state.py` → the shared state shape, clear about what's global vs. per-agent
- a revision-round counter with a hard limit

Expected Behavior

- A normal task flows: supervisor routes to research → analysis → writer → reviewer; if the reviewer rejects it, back to writer (with a limit); the final accepted result comes back with a full log.

- No two agents redundantly call the same tool for the same sub-task.
- The system reports a clear failure (not an endless loop, not a silent bad answer) if revision never settles within the limit.
- If one specialist fails or returns something unusable, the system does one of the three honest things from Error propagation between agents — it never just carries on silently as if nothing happened.

Test Cases

Scenario	Expected
A normal task	Full pipeline finishes, reviewer accepts on the first pass
Writer's first draft is deliberately weak (a test case)	Reviewer rejects it, writer improves it, eventually accepted
Revision never settles (a tricky test case)	Limit is hit, a clear "couldn't agree" report, no endless loop
Task is unclear between two specialists	Supervisor routes to one, and you can explain why from its instructions
One specialist's tool call fails mid-run	The system does one of the three honest options — retries, routes around, or fails clearly — never silently continues with a bad result

Break-It / Debug Preview

- The supervisor routes to the wrong agent.
- Two agents redundantly call the same tool.
- Reviewer and writer never agree.
- One agent's private notes leak into shared state and confuse a later agent.
- A specialist's tool fails, and its bad or empty result quietly flows downstream unnoticed.
- Full debugging drill in 14_debugging_lab.

Interview Topics Preview

- Full trade-off table for every pattern above · defending "why 4 agents and not 1" · shared vs. private state · `Command` vs. the older text-code routing pattern · the three real options for error propagation between agents · redundant-work and state-leaking failures.

□ You Can Now Build Project 4

Doc11 is everything Project 4 needs on top of Project 3. Go to project_4_contentforge_multi_agent/ and start with its **Setup** section — the step-by-step build guide there (not the Build Task summary above) is what you actually follow.

Move On When

Project 4 finishes a full task through all four agents, survives at least two break scenarios, and you can defend the architecture choice without being asked twice. Full details: CURRICULUM.md §4.

Stuck? Ask for **Hint 1** or **Hint 2**. Ask for the full solution only if you say **"Show me the solution."**